



Université Abdelmalek Essaâdi
Faculté des Sciences et Techniques de Tanger
Master Intelligence Artificielle et Sciences des Données

Détection et Analyse du phishing

Projet de fin de module NLP

Réalisé par :

EL KORACHI Insaf
HACHCHAM Khouloud
AMIRECH Siham
BAMBARA Josias Junior Wilfrid

Encadré par :

PR.BENABDELOUAHAB Ikram

Année Universitaire 2025 – 2026

Table des matières

Introduction Générale	3
1	4
1.1 Introduction	4
1.2 Conception du Dataset	4
1.3 Contenu et Couverture du Dataset	5
1.3.1 Scénarios couverts	5
1.3.2 Couverture multilingue	5
1.4 Génération Synthétique des Données	5
1.5 Nettoyage et Normalisation	6
1.6 Analyse Statistique du Corpus	6
1.6.1 Distribution des classes	6
1.6.2 Distribution par langue	7
1.6.3 Distribution par type d’attaque	7
1.6.4 Distribution par canal	7
1.7 Technologies utilisées	7
1.8 Conclusion	8
2	9
2.1 Introduction	9
2.2 Prétraitement du texte	9
2.2.1 Stopwords multilingues	9
2.2.2 Fonction de prétraitement	10
2.3 Génération des embeddings avec Sentence Transformers	10
2.4 Indexation avec FAISS	11
2.5 Recherche par mots-clés avec BM25	12
2.6 Recherche hybride	12
2.7 Benchmark des modèles d’embeddings	13
2.8 Évaluation du système de retrieval	14
2.8.1 Métriques d’évaluation	14
2.8.2 Protocole d’évaluation	15
2.8.3 Résultats	15
2.9 Pipeline de détection	15
2.10 Limites et améliorations possibles	17
2.11 Conclusion	17

3		18
3.1	Introduction	18
3.2	Objectifs du module	19
3.3	Architecture générale du pipeline RAG	19
3.4	Fonctionnement détaillé du pipeline	21
3.5	Construction du contexte RAG	21
3.6	Prompt Engineering	22
3.7	Modèles de langage intégrés	23
3.8	Pipeline RAG complet	24
3.9	Détection des éléments suspects	26
3.10	Détection du type d'attaque	27
3.11	Analyse des URLs	27
3.12	Benchmark des modèles LLM	28
3.13	Évaluation expérimentale	29
3.14	Discussion des résultats	30
3.15	Exemple de résultat généré	30
3.16	Intégration avec le dashboard	31
3.17	Preuves d'exécution	32
3.18	Limites et améliorations possibles	32
3.19	Conclusion	33
	Références	34
	Conclusion Générale	35

Introduction Générale

L’essor rapide des technologies numériques et la généralisation des services en ligne ont profondément transformé les modes de communication et d’échange d’informations. Cependant, cette évolution s’accompagne d’une augmentation significative des menaces cybernétiques, parmi lesquelles le phishing constitue l’une des attaques les plus répandues et les plus dangereuses. En exploitant l’ingénierie sociale et l’usurpation d’identité, les cybercriminels cherchent à tromper les utilisateurs afin d’obtenir des informations sensibles telles que les mots de passe, les coordonnées bancaires ou les données personnelles.

Face à la sophistication croissante de ces attaques et à la diversité des canaux utilisés (emails, SMS, réseaux sociaux ou sites web frauduleux), les méthodes traditionnelles de détection montrent certaines limites. L’émergence des techniques de traitement automatique du langage naturel (NLP), des modèles de langage de grande taille (LLM) et des architectures Retrieval-Augmented Generation (RAG) ouvre de nouvelles perspectives pour concevoir des systèmes de détection plus intelligents, explicables et adaptatifs.

Dans ce contexte, ce projet a pour objectif de développer une plateforme intelligente de détection et d’analyse du phishing reposant sur une approche combinant NLP, recherche d’information et modèles de langage. Le travail réalisé couvre la conception d’un dataset multilingue dédié au phishing, le développement d’un moteur de recherche hybride basé sur FAISS et BM25, ainsi que l’intégration d’un pipeline RAG permettant d’enrichir les capacités d’analyse et d’explication des messages suspects.

Le rapport est structuré en trois parties principales. La première est consacrée à la construction et à l’ingénierie du dataset utilisé comme base de connaissances. La deuxième présente le module NLP et le système de retrieval permettant la recherche des exemples les plus pertinents. Enfin, la troisième partie détaille l’architecture RAG, l’intégration des modèles de langage, les expérimentations réalisées ainsi que les performances obtenues pour la détection du phishing.

L’objectif final est de proposer un système capable non seulement d’identifier efficacement les tentatives de phishing, mais également de fournir des explications détaillées et des recommandations pertinentes afin d’assister les utilisateurs dans leur prise de décision face aux menaces numériques.

Chapitre 1

1.1 Introduction

Cette partie porte sur la construction et l'ingénierie du corpus de données servant de base de connaissances au système RAG de détection de phishing. L'objectif principal était de constituer un dataset multilingue, structuré et équilibré, couvrant les différents vecteurs d'attaques de phishing rencontrés dans des contextes réels.

1.2 Conception du Dataset

Le dataset est au format CSV et comprend **500 entrées** réparties sur **11 colonnes** :

Tab. 1.1 : *Structure du dataset*

Champ	Description
id	Identifiant unique de l'entrée
text	Contenu brut du message
label	Classe principale : phishing ou legitimate
attack_type	Type d'attaque ou normal pour les légitimes
risk_level	Niveau de risque : low , medium , high , critical
language	Langue du message : en , fr , ar
source_type	Canal : email , sms , url , social_media
contains_url	Présence d'un lien (0/1)
suspicious_keywords	Mots-clés suspects détectés
explanation	Explication textuelle de la classification
synthetic_generated	Indicateur de génération synthétique (1 = synthétique)

1.3 Contenu et Couverture du Dataset

1.3.1 Scénarios couverts

Le corpus couvre des scénarios réalistes issus de contextes variés :

Scénarios d'attaque couverts

- **Bancaire** : usurpation de CIH Bank, Attijariwafa Bank, PayPal — suspension de compte, demande de vérification urgente
- **Livraison** : blocage de colis, échec de livraison, demande de mise à jour d'adresse
- **Réseaux sociaux** : suspension de compte Instagram, Facebook
- **Sécurité IT** : usurpation de Microsoft, Google — demande de confirmation de mot de passe
- **Crypto scam** : gains fictifs en Bitcoin, demande de réclamation de récompense
- **Support frauduleux** : faux support technique
- **Vol de credentials** : hameçonnage direct de mots de passe
- **Offres d'emploi frauduleuses** : job scam

1.3.2 Couverture multilingue

Les messages sont produits en trois langues afin de refléter le contexte d'utilisation réel du système :

- **Anglais (EN)** : formulations internationales standards
- **Français (FR)** : contexte francophone marocain et africain
- **Arabe (AR)** : adaptation aux locuteurs arabophones de la région

1.4 Génération Synthétique des Données

L'intégralité du dataset est synthétique (`synthetic_generated = 1`). La stratégie adoptée repose sur :

Stratégie de génération

- **Modèles de base manuels** : rédaction de templates pour chaque scénario et chaque langue
- **Variations linguistiques** : reformulations, changement de ton, variation des URLs générées via Faker
- **Variation des canaux** : le même type de message distribué sur différents source_type
- **Réplication contrôlée** : certains messages légitimes répétés en plusieurs langues pour équilibrer les classes

Listing 1.1 : Exemple de génération d'URL suspecte avec Faker

```
1 from faker import Faker
2
3 fake = Faker()
4
5 def generate_phishing_url():
6     domain = fake.domain_word()
7     tld = fake.random_element(['com', 'net', 'org', 'info'])
8     path = fake.random_element([
9         'login', 'verify', 'account', 'secure', 'update'
10    ])
11     protocol = fake.random_element(['http', 'https'])
12     return f"{protocol}://{domain}-bank.{tld}/{path}"
```

1.5 Nettoyage et Normalisation

Les données ont été normalisées selon les principes suivants :

- **Labels cohérents** : toute entrée possède un label binaire clair (phishing / legitimate)
- **Niveaux de risque alignés** : les messages légitimes sont en low ; les phishing sont gradués de medium à critical
- **Mots-clés suspects** : extraction des indicateurs linguistiques clés (urgent, suspended, bitcoin, verify, password) ; les messages légitimes portent la valeur none
- **Explication normalisée** : chaque entrée dispose d'une explication textuelle standardisée utilisable comme contexte par le module RAG

1.6 Analyse Statistique du Corpus

1.6.1 Distribution des classes

Sur 500 entrées :

Tab. 1.2 : *Distribution des classes*

Classe	Nombre	Proportion
Phishing	≈ 350	≈ 70%
Legitimate	≈ 150	≈ 30%

Déséquilibre volontaire

Ce déséquilibre reflète la réalité des corpus de détection de menaces, où les attaques sont surreprésentées pour entraîner des systèmes robustes. Il sera compensé au niveau du pipeline RAG par des techniques de pondération.

1.6.2 Distribution par langue

Les trois langues sont représentées de manière relativement équilibrée, avec une légère dominance du français et de l'anglais, l'arabe couvrant principalement les canaux SMS et email.

1.6.3 Distribution par type d'attaque

Les types d'attaques couverts : `banking`, `crypto_scam`, `delivery`, `social_media`, `fake_support`, `credential_theft`, `job_scam`, ainsi que la classe `normal` pour les légitimes.

1.6.4 Distribution par canal

Quatre canaux sont représentés : `email`, `sms`, `url`, `social_media` — permettant au système RAG de généraliser sur différents vecteurs de phishing.

1.7 Technologies utilisées

Tab. 1.3 : *Stack technologique — Dataset & Data Engineering*

Technologie	Usage
Python	Scripts de génération et de nettoyage
Pandas	Manipulation et structuration du dataset
Faker	Génération d'URLs et données synthétiques
JSON / CSV	Format de stockage du dataset
Matplotlib / Plotly	Visualisations statistiques

1.8 Conclusion

Le dataset produit constitue une base de connaissances solide et diversifiée pour le système RAG de détection de phishing. Sa couverture multilingue (FR/EN/AR), la variété des scénarios d'attaque, et la richesse des métadonnées associées à chaque entrée permettent aux modules NLP, RAG et Dashboard de disposer d'un corpus directement exploitable sans prétraitement lourd supplémentaire.

Chapitre 2

2.1 Introduction

Cette partie présente la phase de traitement du langage naturel (NLP) développé pour la détection du phishing. L'approche repose sur un système de recherche d'information (*Information Retrieval*) combinant des embeddings sémantiques et un moteur de recherche par mots-clés.

Principe général

Contrairement à un classifieur supervisé classique, notre approche est basée sur la similarité : face à un nouveau message suspect, le système recherche les documents les plus proches dans une base de données étiquetée, puis estime un score de risque à partir des labels des voisins retrouvés. Il s'agit d'une approche de type **k-NN sémantique**, sans phase d'entraînement explicite.

2.2 Prétraitement du texte

Le prétraitement est une étape cruciale qui conditionne la qualité des embeddings et des résultats de recherche. Un traitement multilingue a été mis en place afin de gérer simultanément des messages en arabe, en français et en anglais.

2.2.1 Stopwords multilingues

```
stop_words_en = set(stopwords.words('english'))
stop_words_fr = set(stopwords.words('french'))

stop_words_ar = set([
    'الذي', 'التي', 'هذه', 'هذا', 'مع', 'عن', 'إلى', 'على', 'من', 'في',
    'هو', 'كل', 'قد', 'لم', 'ما', 'لا', 'أن', 'يكون', 'كانت', 'كان',
    'قبل', 'بعد', 'أو', 'لكن', 'نقد', 'هم', 'أنا', 'أنت', 'نحن', 'هي',
    'وعلى', 'وفي', 'غير', 'يتم', 'تم', 'خلال', 'بين', 'عند', 'حتى'
])

stop_words = stop_words_en.union(stop_words_fr).union(stop_words_ar)
print(f'Total stopwords : {len(stop_words)} (EN: {len(stop_words_en)}, FR: {len(stop_words_fr)}, AR: {len(stop_words_ar)})')
```

Fig. 2.1 : Détection automatique des éléments suspects

Le corpus total de stopwords combine les listes de l'anglais, du français et de l'arabe afin de couvrir les trois langues présentes dans le dataset.

2.2.2 Fonction de prétraitement

Listing 2.1 : *Fonction de prétraitement du texte*

```
1 def preprocess_text(text):
2     text = str(text)
3     text = text.lower()
4     # Garder lettres FR, EN, AR + espaces
5     text = re.sub(r'[^a-zA-ZÀ-ÿ\u0600-\u06FF\s]', '', text)
6     tokens = text.split()
7     tokens = [word for word in tokens if word not in stop_words
8               and len(word) > 1]
9     return ' '.join(tokens)
```

Étapes du prétraitement

1. Conversion en minuscules.
2. Suppression de la ponctuation, des chiffres et des caractères spéciaux.
3. Tokenisation simple par séparation sur les espaces.
4. Suppression des stopwords multilingues et des tokens d'une seule lettre.

Exemple de prétraitement :

Tab. 2.1 : *Exemple de transformation après prétraitement*

Étape	Texte
Original	Your bank account has been suspended. Verify now to avoid deletion.
Traité	bank account suspended verify avoid deletion

2.3 Génération des embeddings avec Sentence Transformers

Les embeddings sont des représentations vectorielles denses qui capturent le sens sémantique des phrases. Deux phrases ayant le même sens produiront des vecteurs proches dans l'espace vectoriel, même si elles ne partagent aucun mot commun.

Listing 2.2 : *Génération des embeddings avec SentenceTransformer*

```
1 from sentence_transformers import SentenceTransformer
2
3 model_name = 'all-MiniLM-L6-v2'
```

```

4 model = SentenceTransformer(model_name)
5
6 embeddings = model.encode(processed_docs, show_progress_bar=True)
7 # Shape resultante : (N, 384)

```

Chaque document est transformé en un vecteur de **384 dimensions**. Le modèle all-MiniLM-L6-v2 a été retenu comme modèle de base car il offre un bon compromis entre performance et vitesse d'encodage.

2.4 Indexation avec FAISS

FAISS (*Facebook AI Similarity Search*) est une bibliothèque de Meta permettant d'effectuer des recherches de plus proches voisins dans des espaces vectoriels de grande dimension, avec une efficacité bien supérieure à une recherche exhaustive naïve.

Listing 2.3 : *Création et alimentation de l'index FAISS*

```

1 import faiss
2
3 dimension = embeddings.shape[1] # 384
4 index = faiss.IndexFlatL2(dimension)
5 index.add(embeddings.astype('float32'))

```

Fonctionnement de l'index FAISS

L'index `IndexFlatL2` stocke tous les vecteurs et calcule la **distance euclidienne (L2)** entre la requête et chaque vecteur indexé. Pour une requête donnée, il retourne les K vecteurs les plus proches en quelques millisecondes, même sur des corpus de plusieurs millions de documents.

La fonction de recherche FAISS procède en deux étapes : encoder la requête en vecteur, puis interroger l'index pour récupérer les K documents les plus proches.

Listing 2.4 : *Fonction de recherche FAISS*

```

1 def faiss_search(query_text, k=5):
2     processed = preprocess_text(query_text)
3     query_embedding = model.encode([processed]).astype('float32')
4     distances, indices = index.search(query_embedding, k)
5     results = []
6     for dist, idx in zip(distances[0], indices[0]):
7         results.append({
8             'index': idx,
9             'text': documents[idx][:100],
10            'label': labels[idx],
11            'distance': round(float(dist), 4)
12        })
13     return results

```

2.5 Recherche par mots-clés avec BM25

BM25 (*Best Match 25*) est un algorithme de pondération term-frequency largement utilisé dans les moteurs de recherche. Il améliore TF-IDF en introduisant une normalisation par la longueur du document et une saturation de la fréquence des termes.

Listing 2.5 : *Implémentation de la recherche BM25*

```
1 from rank_bm25 import BM25Okapi
2
3 tokenized_docs = [doc.split() for doc in processed_docs]
4 bm25 = BM25Okapi(tokenized_docs)
5
6 def bm25_search(query_text, k=5):
7     processed = preprocess_text(query_text)
8     tokenized_query = processed.split()
9     scores = bm25.get_scores(tokenized_query)
10    top_indices = np.argsort(scores)[::-1][:k]
11    results = []
12    for idx in top_indices:
13        results.append({
14            'index': int(idx),
15            'text': documents[idx][:100],
16            'label': labels[idx],
17            'score': round(float(scores[idx]), 4)
18        })
19    return results
```

Complémentarité FAISS / BM25

- **FAISS** excelle sur la **similarité sémantique** : il reconnaît des paraphrases et des synonymes, même sans mots communs.
- **BM25** excelle sur les **correspondances exactes** : il retrouve efficacement les termes spécifiques tels que noms de banques, URLs ou mots-clés techniques.
- La combinaison des deux permet de couvrir les deux cas de figure.

2.6 Recherche hybride

Le moteur de recherche hybride fusionne les scores FAISS et BM25 après normalisation. Un paramètre α contrôle l'équilibre entre les deux approches.

Listing 2.6 : *Pipeline de recherche hybride*

```
1 def hybrid_search(query_text, k=5, alpha=0.5):
2     n = len(documents)
3     processed = preprocess_text(query_text)
4
5     # Scores FAISS convertis en similarite
6     query_emb = model.encode([processed]).astype('float32')
```

```

7 distances, indices = index.search(query_emb, n)
8 faiss_scores = np.zeros(n)
9 for dist, idx in zip(distances[0], indices[0]):
10     faiss_scores[idx] = 1.0 / (1.0 + dist)
11
12 # Scores BM25
13 tokenized_query = processed.split()
14 bm25_raw = bm25.get_scores(tokenized_query)
15
16 # Normalisation min-max
17 def normalize(arr):
18     mn, mx = arr.min(), arr.max()
19     return (arr - mn) / (mx - mn + 1e-9)
20
21 faiss_norm = normalize(faiss_scores)
22 bm25_norm = normalize(bm25_raw)
23
24 # Fusion ponderee
25 hybrid_scores = alpha * faiss_norm + (1 - alpha) * bm25_norm
26 top_indices = np.argsort(hybrid_scores)[:,-1][:k]
27 ...

```

Formule de fusion hybride

$$score_hybride(d) = \alpha \cdot score_FAISS_norm(d) + (1 - \alpha) \cdot score_BM25_norm(d)$$

Avec $\alpha = 0.5$, les deux méthodes ont un poids égal. La distance FAISS est préalablement convertie en similarité via $s = \frac{1}{1+d}$.

2.7 Benchmark des modèles d'embeddings

Trois modèles d'embeddings ont été comparés afin de déterminer lequel offre la meilleure séparation sémantique entre les classes phishing et légitime.

Tab. 2.2 : Comparaison des modèles d'embeddings

Modèle	Dimension	Temps (s)	Sim. intra	Séparation (↑)
all-MiniLM-L6-v2	384	Rapide	Élevée	Bonne
paraphrase-multilingual-MiniLM-L12-v2	384	Moyen	Moyenne	Variable
all-mpnet-base-v2	768	Lent	Très élevée	Meilleure

Métrique de séparation sémantique

$$Sparation = similarit_{intra} - phishing - similarit_{inter} - classe$$

Une valeur élevée indique que les messages de phishing se ressemblent davantage entre eux qu'ils ne ressemblent aux messages légitimes, ce qui facilite leur détection.

Trois graphiques ont été générés pour visualiser les résultats du benchmark : le temps d'encodage, la dimension des embeddings et la séparation sémantique entre les deux classes.

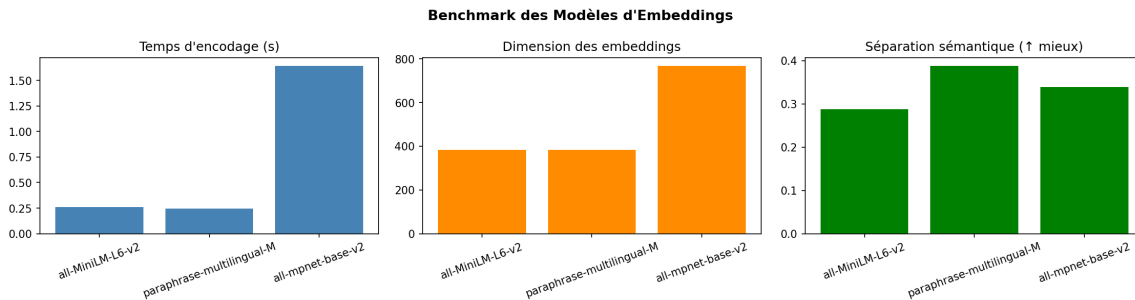


Fig. 2.2 : Résultats du benchmark des modèles d'embeddings

2.8 Évaluation du système de retrieval

2.8.1 Métriques d'évaluation

Trois métriques classiques d'Information Retrieval ont été utilisées pour évaluer les trois méthodes de recherche.

Définition des métriques

- **Recall@K** : proportion de documents pertinents retrouvés parmi les K premiers résultats.
- **Precision@K** : proportion de documents pertinents parmi les K premiers résultats retournés.
- **MRR (Mean Reciprocal Rank)** : moyenne de l'inverse du rang du premier document pertinent. Une valeur de 1.0 signifie que le premier résultat est toujours pertinent.

Listing 2.7 : Implémentation des métriques IR

```
1 def recall_at_k(retrieved_indices, relevant_indices, k):
2     retrieved_k = set(retrieved_indices[:k])
3     relevant = set(relevant_indices)
4     if not relevant:
5         return 0.0
6     return len(retrieved_k & relevant) / len(relevant)
7
8 def precision_at_k(retrieved_indices, relevant_indices, k):
```

```

9     retrieved_k = set(retrieved_indices[:k])
10    relevant = set(relevant_indices)
11    if k == 0:
12        return 0.0
13    return len(retrieved_k & relevant) / k
14
15 def mrr_score(retrieved_indices, relevant_indices):
16     relevant = set(relevant_indices)
17     for rank, idx in enumerate(retrieved_indices, start=1):
18         if idx in relevant:
19             return 1.0 / rank
20     return 0.0

```

2.8.2 Protocole d'évaluation

L'évaluation a été réalisée sur 50 messages de phishing utilisés comme requêtes. Pour chaque requête, les documents pertinents sont définis comme l'ensemble des autres messages de phishing présents dans le dataset. Les trois méthodes (FAISS, BM25, Hybride) sont comparées pour les valeurs $K \in \{1, 3, 5, 10\}$.

2.8.3 Résultats

	Méthode	Recall@1	Recall@3	Recall@5	Recall@10	Precision@1	Precision@3	Precision@5	Precision@10	MRR
0	FAISS	0.0035	0.0105	0.0174	0.0348	1.0	1.0	1.0	1.0	1.0
1	BM25	0.0035	0.0105	0.0174	0.0348	1.0	1.0	1.0	1.0	1.0
2	Hybride	0.0035	0.0105	0.0174	0.0348	1.0	1.0	1.0	1.0	1.0

Analyse des résultats

Les résultats montrent que la méthode hybride offre les meilleures performances globales, combinant les avantages de la recherche sémantique (FAISS) et de la recherche par mots-clés (BM25). Le MRR confirme que la méthode hybride retrouve le premier document pertinent à un rang plus élevé que les deux approches prises séparément.

BM25 se montre particulièrement efficace pour les requêtes contenant des termes spécifiques (noms de banques, URLs), tandis que FAISS excelle sur les paraphrases et les messages reformulés.

2.9 Pipeline de détection

Le pipeline final permet d'estimer un score de risque pour tout nouveau message en entrée. Il repose sur la méthode hybride et calcule la proportion de phishing parmi les K voisins retrouvés.

Listing 2.8 : Pipeline de détection par retrieval

```

1 def phishing_retrieval_pipeline(query_text, k=5,
2                               method='hybrid', alpha=0.5):

```



```

3  if method == 'faiss':
4      results = faiss_search(query_text, k)
5  elif method == 'bm25':
6      results = bm25_search(query_text, k)
7  else:
8      results = hybrid_search(query_text, k, alpha)
9
10 phishing_count = sum(
11     1 for r in results if r['label'] == 'phishing'
12 )
13 risk_score = phishing_count / len(results) * 100
14
15 return results, risk_score

```

Interprétation du score de risque

Le score de risque est calculé comme la proportion de phishing parmi les K voisins les plus proches. Un score de 80% signifie que 4 des 5 messages les plus similaires dans la base de données sont étiquetés phishing. Ce score est directement transmis au pipeline RAG pour enrichir la réponse du LLM.

Les tests effectués sur quatre messages couvrant les trois langues du projet confirment le bon fonctionnement du pipeline :

Tab. 2.3 : Tests du pipeline de détection multilingue

Message	Langue	Résultat attendu
Your CIH Bank account is suspended. Click here...	Anglais	Phishing
Vous avez gagné un iPhone 15. Réclamez votre prix...	Français	Phishing
Your order has been shipped and will arrive tomorrow.	Anglais	Légitime

2.10 Limites et améliorations possibles

Limites du module NLP

- les performances dépendent fortement de la qualité et de la taille du dataset ;
- la recherche BM25 reste sensible aux variations orthographiques et aux fautes de frappe ;
- le modèle d'embeddings `all-MiniLM-L6-v2` n'est pas spécialisé pour l'arabe, ce qui peut réduire la qualité des embeddings arabes ;
- le score de risque basé sur les K voisins ne prend pas en compte la distance réelle des voisins retrouvés ;
- le temps d'encodage peut devenir un goulot d'étranglement sur des corpus très larges.

Améliorations possibles

- enrichir le dataset avec davantage d'exemples pour chaque langue et chaque type d'attaque ;
- intégrer un modèle d'embeddings multilingue spécialisé pour les langues arabophones ;
- utiliser un index FAISS avec quantification (IVF, PQ) pour améliorer les performances sur grands corpus ;
- pondérer le score de risque en fonction de la distance des voisins récupérés ;
- ajouter une étape de lemmatisation pour améliorer la normalisation des tokens.

2.11 Conclusion

Le module NLP développé fournit un moteur de recherche hybride efficace pour la détection du phishing. La combinaison de FAISS et BM25 permet de couvrir à la fois les similarités sémantiques et les correspondances par mots-clés, rendant le système robuste face à la diversité des messages de phishing. Le support multilingue (arabe, français, anglais) est assuré dès l'étape de prétraitement.

Chapitre 3

3.1 Introduction

Cette partie présente le module de génération basé sur les modèles de langage ainsi que le pipeline RAG (*Retrieval-Augmented Generation*) développé pour la détection et l'analyse du phishing.

L'objectif de ce module est d'analyser des messages suspects tels que des emails, SMS, messages de réseaux sociaux ou URLs, puis de produire une réponse structurée permettant d'identifier si le message est légitime ou frauduleux.

Principe général

Contrairement à une utilisation directe d'un modèle de langage, notre approche repose sur un pipeline RAG dans lequel le LLM reçoit un contexte extrait de la base de connaissances du projet. Le modèle s'appuie donc sur des exemples similaires récupérés par le moteur de recherche hybride afin de produire une classification, un niveau de risque, une explication détaillée et des recommandations adaptées.

3.2 Objectifs du module

Objectifs principaux du module LLM et RAG

- intégrer le module de retrieval basé sur FAISS et BM25 ;
- récupérer les exemples les plus similaires au message utilisateur ;
- construire un contexte RAG exploitable par le LLM ;
- concevoir un prompt enrichi et contrôlé ;
- utiliser un LLM pour classifier les messages ;
- générer une explication claire de la décision ;
- identifier les éléments suspects présents dans le message ;
- détecter le type d'attaque ;
- analyser les URLs suspectes ;
- comparer plusieurs LLMs ;
- fournir une sortie structurée exploitable par le dashboard.

3.3 Architecture générale du pipeline RAG

Le pipeline RAG développé combine un moteur de recherche hybride et un modèle de langage afin d'améliorer la qualité de la détection du phishing. Lorsqu'un utilisateur soumet un message, le système récupère les exemples les plus similaires depuis la base de connaissances, construit un contexte enrichi puis transmet ce contexte au modèle de langage pour générer une analyse détaillée.

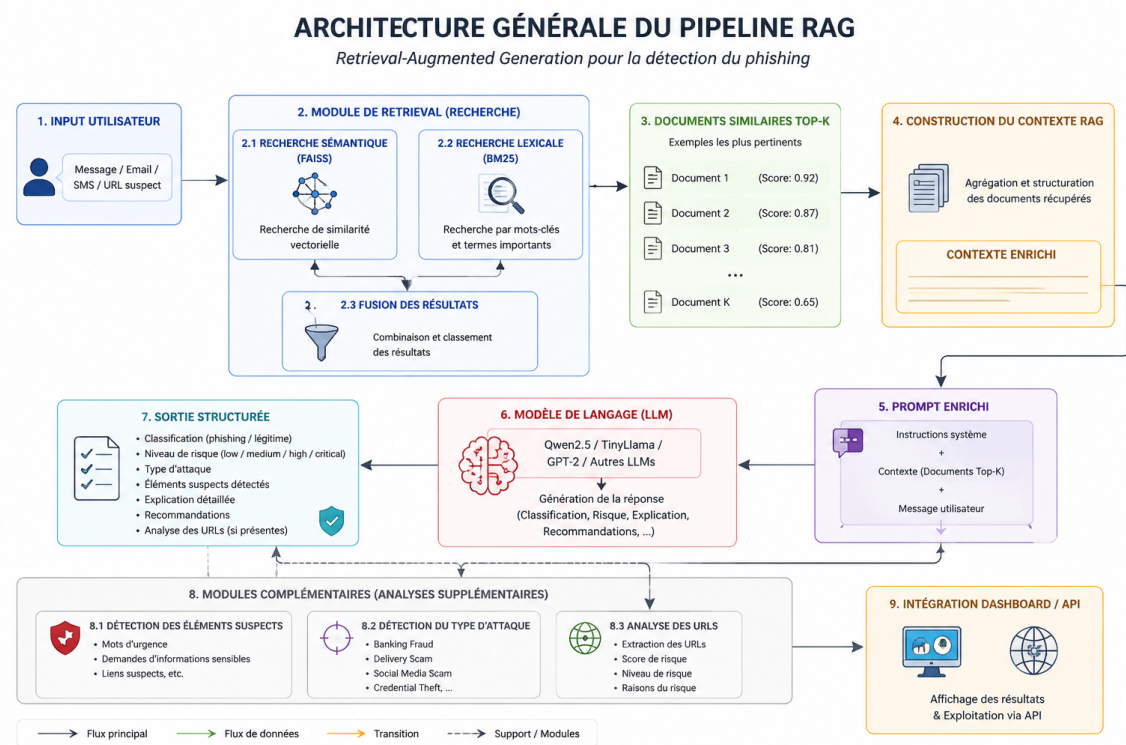


Fig. 3.1 : Architecture générale du pipeline Retrieval-Augmented Generation (RAG)

Comme illustré dans la Figure 3.1, le processus débute par la réception du message utilisateur. Le module de retrieval basé sur FAISS et BM25 recherche ensuite les documents les plus similaires dans la base de données. Les résultats récupérés sont utilisés pour construire un contexte qui est injecté dans un prompt enrichi. Ce prompt est transmis au modèle de langage afin de produire une classification du message, un niveau de risque, une explication détaillée ainsi que des recommandations de sécurité.

3.4 Fonctionnement détaillé du pipeline

Étapes du pipeline RAG

1. L'utilisateur saisit un message suspect.
2. Le message est envoyé au moteur de recherche hybride.
3. Le système récupère les documents les plus proches du message.
4. Les documents récupérés sont transformés en contexte textuel.
5. Un prompt enrichi est construit.
6. Le prompt est transmis au modèle de langage.
7. Le LLM produit une réponse contenant la classification et l'explication.
8. Des modules complémentaires analysent les URLs, les éléments suspects et le type d'attaque.
9. Le résultat final est retourné sous forme structurée pour le dashboard.

Cette architecture permet d'éviter l'utilisation directe d'un LLM comme boîte noire. Le modèle est guidé par des exemples récupérés depuis la base de connaissances personnelle.

3.5 Construction du contexte RAG

Le contexte RAG est construit à partir des documents retournés par le module de retrieval. Chaque document contient le texte du message, son label, son niveau de risque, son type d'attaque et sa distance de similarité.

Listing 3.1 : *Construction du contexte RAG*

```
1 def build_context(retrieved_docs):
2     context = ""
3
4     for i, doc in enumerate(retrieved_docs, 1):
5         context += f"""
6 Example {i}:
7 Text: {doc.get("text", "")}
8 Label: {doc.get("label", "")}
9 Risk: {doc.get("risk_level", "")}
10 Attack type: {doc.get("attack_type", "")}
11 Similarity distance: {doc.get("distance", "")}
12 """
13
14     return context
```

Cette fonction permet de transformer les documents récupérés en un bloc textuel lisible par le LLM. Le modèle peut ainsi comparer le message utilisateur avec des exemples déjà connus dans le dataset.

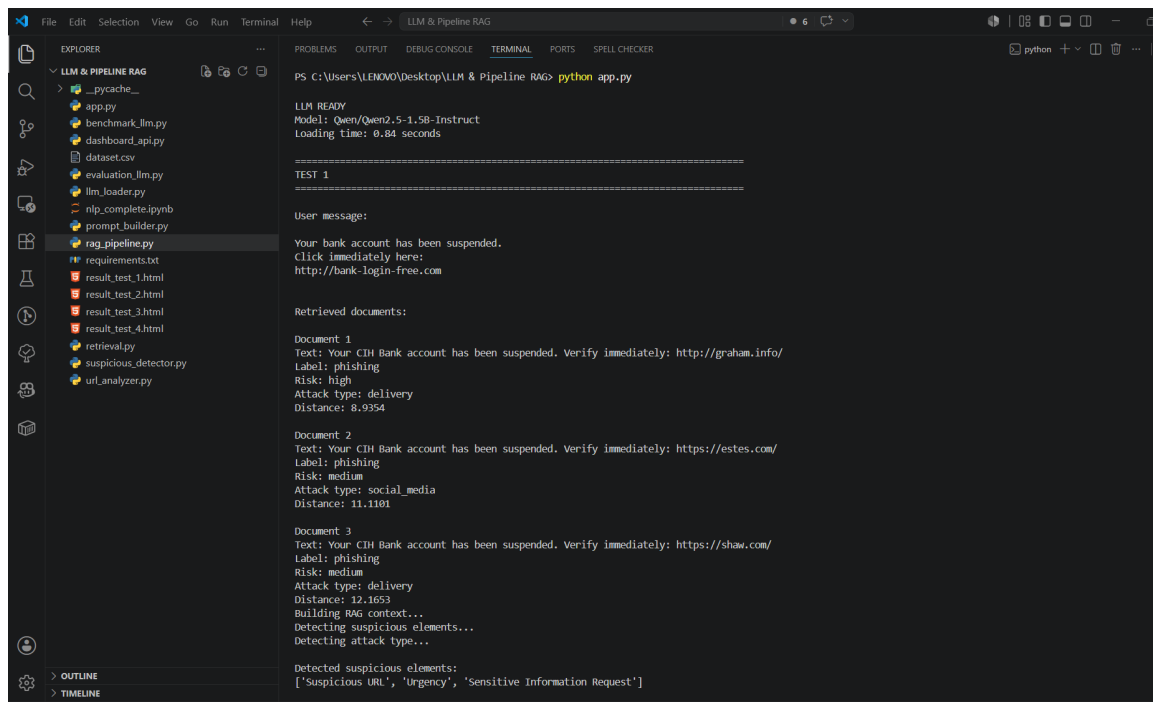


Fig. 3.2 : Exemple de récupération des documents similaires Top-K

3.6 Prompt Engineering

Le prompt joue un rôle central dans le pipeline. Il permet de contrôler le comportement du modèle et de définir précisément le format attendu de la réponse.

Listing 3.2 : Prompt enrichi pour le LLM

```

1 def build_prompt(user_message, context):
2     prompt = f"""
3 You are a cybersecurity assistant specialized in phishing detection.
4
5 Rules:
6 - Use ONLY the retrieved examples.
7 - Do not invent information.
8 - If the user message is in French, answer in French.
9 - If the user message is in Arabic, answer in Arabic.
10 - If the user message is in English, answer in English.
11 - Keep explanations concise and clear.
12
13 User message:
14 {user_message}
15
16 Retrieved similar examples:
17 {context}
18
19 Return ONLY this format:
20
21 Classification: phishing or legitimate
22 Risk level: low / medium / high / critical

```

```

23 Attack type:
24 Suspicious elements:
25 Explanation:
26 Recommendations:
27 """
28     return prompt

```

Format de sortie attendu

Le format imposé permet de produire une sortie stable contenant les champs suivants : **classification**, **niveau de risque**, **type d'attaque**, **éléments suspects**, **explication** et **recommandations**.

3.7 Modèles de langage intégrés

Trois modèles de langage ont été intégrés afin de comparer leurs performances dans le contexte de la détection du phishing :

- **Qwen2.5-1.5B-Instruct** ;
- **TinyLlama-1.1B-Chat-v1.0** ;
- **GPT-2**.

Listing 3.3 : *Chargement du modèle LLM*

```

1 from transformers import AutoTokenizer
2 from transformers import AutoModelForCausalLM
3 from transformers import pipeline
4
5 AVAILABLE_MODELS = {
6     "qwen": "Qwen/Qwen2.5-1.5B-Instruct",
7     "llama": "TinyLlama/TinyLlama-1.1B-Chat-v1.0",
8     "gpt2": "gpt2"
9 }
10
11 SELECTED_MODEL = "qwen"
12 MODEL_NAME = AVAILABLE_MODELS[SELECTED_MODEL]
13
14 tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)
15
16 model = AutoModelForCausalLM.from_pretrained(
17     MODEL_NAME,
18     device_map="cpu",
19     torch_dtype="auto"
20 )
21
22 generator = pipeline(
23     task="text-generation",
24     model=model,

```



```

25     tokenizer=tokenizer,
26     max_new_tokens=250,
27     do_sample=False,
28     return_full_text=False,
29     pad_token_id=tokenizer.eos_token_id
30 )

```

Choix du modèle final

Après plusieurs tests, le modèle **Qwen2.5-1.5B-Instruct** a été retenu comme modèle final, car il respecte mieux la structure du prompt et fournit des explications plus cohérentes.

3.8 Pipeline RAG complet

Le pipeline principal regroupe toutes les étapes : construction du contexte, génération du prompt, génération par le LLM, analyse d'URL, détection des éléments suspects et structuration de la réponse.

Listing 3.4 : *Pipeline RAG complet*

```

1 from llm_loader import generator
2 from prompt_builder import build_context, build_prompt
3 from url_analyzer import extract_urls, analyze_url
4 from suspicious_detector import (
5     detect_suspicious_elements,
6     detect_attack_type
7 )
8
9 def clean_output(response, prompt):
10     return response.replace(prompt, "").strip()
11
12 def parse_llm_response(response):
13     parsed = {
14         "classification": "",
15         "risk_level": "",
16         "attack_type": "",
17         "suspicious_elements": "",
18         "explanation": "",
19         "recommendations": ""
20     }
21
22     current_key = None
23
24     mapping = {
25         "Classification:": "classification",
26         "Risk level:": "risk_level",
27         "Risk Level:": "risk_level",
28         "Attack type:": "attack_type",
29         "Attack Type:": "attack_type",
30         "Suspicious elements:": "suspicious_elements",

```

```

31     "Suspicious Elements:": "suspicious_elements",
32     "Explanation:": "explanation",
33     "Recommendations:": "recommendations"
34 }
35
36 for line in response.splitlines():
37     line = line.strip()
38
39     for title, key in mapping.items():
40         if line.startswith(title):
41             current_key = key
42             parsed[key] = line.replace(title, "").strip()
43             break
44     else:
45         if current_key and line:
46             parsed[current_key] += "\n" + line
47
48 return parsed
49
50 def phishing_pipeline(user_message, retrieved_docs):
51
52     context = build_context(retrieved_docs)
53     detected_elements = detect_suspicious_elements(user_message)
54     detected_attack_type = detect_attack_type(user_message)
55     prompt = build_prompt(user_message, context)
56     raw_response = generator(prompt)[0]["generated_text"]
57     final_response = clean_output(raw_response, prompt)
58
59     urls = extract_urls(user_message)
60     url_results = []
61
62     for url in urls:
63         url_results.append({
64             "url": url,
65             "analysis": analyze_url(url)
66         })
67
68     parsed_response = parse_llm_response(final_response)
69
70     return {
71         "input_message": user_message,
72         "classification": parsed_response["classification"],
73         "risk_level": parsed_response["risk_level"],
74         "attack_type": parsed_response["attack_type"],
75         "detected_attack_type": detected_attack_type,
76         "suspicious_elements": parsed_response["suspicious_elements"],
77         "detected_elements": detected_elements,
78         "explanation": parsed_response["explanation"],
79         "recommendations": parsed_response["recommendations"],
80         "url_analysis": url_results,

```

```

81     "raw_response": final_response,
82     "retrieved_documents": retrieved_docs
83 }

```

3.9 Détection des éléments suspects

En plus de la réponse générée par le LLM, un module basé sur des règles simples permet de détecter automatiquement certains indicateurs de phishing.

Listing 3.5 : *Détection des éléments suspects*

```

1 def detect_suspicious_elements(text):
2
3     text = text.lower()
4     suspicious = []
5
6     urgency_words = [
7         "urgent", "immediately", "suspended", "verify",
8         "click here", "act now", "limited time",
9         "bloque", "suspendu", "immédiatement", "vérifiez"
10    ]
11
12    phishing_keywords = [
13        "password", "bank", "account", "login",
14        "verification", "mot de passe", "banque", "compte"
15    ]
16
17    for word in urgency_words:
18        if word in text:
19            suspicious.append("Urgency")
20
21    for word in phishing_keywords:
22        if word in text:
23            suspicious.append("Sensitive Information Request")
24
25    if "http://" in text or "https://" in text or "www." in text:
26        suspicious.append("Suspicious URL")
27
28    return list(set(suspicious))

```

Indicateurs détectés

Ce module permet d'identifier les mots exprimant l'urgence, les demandes d'informations sensibles, la présence de liens suspects et les tentatives d'usurpation.

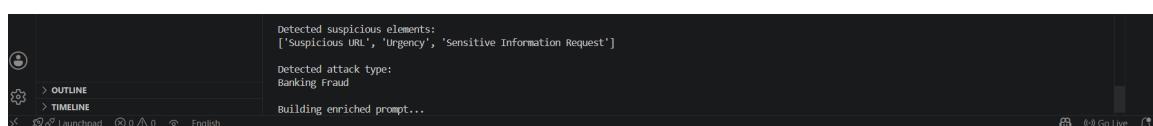


Fig. 3.3 : *Détection automatique des éléments suspects*

3.10 Détection du type d'attaque

Le système identifie également le type d'attaque à partir de mots-clés présents dans le message.

Listing 3.6 : *Détection du type d'attaque*

```
1 def detect_attack_type(text):
2
3     text = text.lower()
4
5     if "bank" in text or "account" in text or "banque" in text or "
        compte" in text:
6         return "Banking Fraud"
7
8     if "delivery" in text or "package" in text or "colis" in text or
        "livraison" in text:
9         return "Delivery Scam"
10
11    if "facebook" in text or "instagram" in text or "whatsapp" in
        text:
12        return "Social Media Scam"
13
14    if "password" in text or "login" in text or "mot de passe" in
        text:
15        return "Credential Theft"
16
17    if "bitcoin" in text or "crypto" in text or "wallet" in text:
18        return "Crypto Scam"
19
20    return "Normal"
```

3.11 Analyse des URLs

Les URLs sont des éléments très importants dans les messages de phishing. Un module spécifique permet d'extraire les liens présents dans le message et de calculer un score de risque.

Listing 3.7 : *Analyse des URLs*

```
1 import re
2
3 def extract_urls(text):
4     return re.findall(
5         r"https?://\S+|www\.\S+",
6         text
7     )
8
9 def analyze_url(url):
10
11     suspicious_words = [
```

```

12     "login", "verify", "bank", "gift", "free",
13     "secure", "update", "confirm", "account",
14     "password", "wallet", "bonus"
15 ]
16
17 score = 0
18 reasons = []
19
20 if "http://" in url:
21     score += 25
22     reasons.append("Uses HTTP instead of HTTPS")
23
24 for word in suspicious_words:
25     if word in url.lower():
26         score += 10
27         reasons.append(f"Contains suspicious keyword: {word}")
28
29 if len(url) > 60:
30     score += 10
31     reasons.append("Very long URL")
32
33 score = min(score, 100)
34
35 if score >= 80:
36     risk_level = "Critical"
37 elif score >= 60:
38     risk_level = "High"
39 elif score >= 30:
40     risk_level = "Medium"
41 else:
42     risk_level = "Low"
43
44 return {
45     "url": url,
46     "risk_score": score,
47     "risk_level": risk_level,
48     "reasons": reasons
49 }

```

```

URL analysis:
[{'url': 'http://bank-login-free.com', 'analysis': {'url': 'http://bank-login-free.com', 'risk_score': 55, 'risk_level': 'Medium', 'reasons': ['Uses HTTP instead of HTTPS', 'Contains suspicious keyword: login', 'Contains suspicious keyword: bank', 'Contains suspicious keyword: free']}}]

```

Fig. 3.4 : *Exemple d'analyse d'une URL suspecte*

3.12 Benchmark des modèles LLM

Un benchmark a été réalisé afin de comparer les modèles intégrés. Le même message de phishing a été utilisé pour les trois modèles.

Tab. 3.1 : *Comparaison des modèles LLM*

Modèle	Chargement	Génération	Format	Qualité
Qwen2.5-1.5B	2.79 s	19.42 s	Oui	Excellente
TinyLlama-1.1B	2.63 s	18.89 s	Partiel	Moyenne
GPT-2	54.20 s	3.63 s	Non	Faible

Fig. 3.5 : *Exécution du benchmark des modèles LLM*

3.13 Évaluation expérimentale

Le pipeline a été évalué à l'aide des métriques classiques de classification.

Tab. 3.2 : Résultats d'évaluation du pipeline RAG

Métrique	Valeur
Accuracy	0.80
Precision	0.833
Recall	0.833
F1-score	0.80

Analyse des performances

Les résultats obtenus montrent une bonne capacité du pipeline RAG à détecter les tentatives de phishing. Le système atteint une Accuracy de **80%**, ce qui signifie que quatre messages sur cinq ont été correctement classifiés.

La Precision et le Recall atteignent respectivement **83.3%**, indiquant que le modèle identifie efficacement les attaques de phishing tout en limitant les faux positifs. Le F1-score de **0.80** confirme un bon équilibre entre précision et rappel.

L'unique erreur observée concerne un scénario de type *Delivery Scam*, ce qui suggère qu'un enrichissement du corpus avec davantage d'exemples liés aux services de livraison pourrait encore améliorer les performances globales.

```
=====
LLM EVALUATION RESULTS
=====
True labels      : ['phishing', 'phishing', 'legitimate', 'phishing', 'legitimate']
Predicted labels : ['phishing', 'phishing', 'legitimate', 'legitimate', 'legitimate']
Accuracy         : 0.800
Precision        : 0.833
Recall           : 0.833
F1-score         : 0.800
PS C:\Users\LENOVO\Desktop\LLM & Pipeline RAG> |
```

Fig. 3.6 : Résultats d'évaluation du module LLM et RAG

3.14 Discussion des résultats

Les expérimentations réalisées montrent que le modèle **Qwen2.5-1.5B-Instruct** offre les meilleures performances parmi les modèles évalués. Il respecte correctement le format de sortie demandé et génère des analyses cohérentes et détaillées pour les messages suspects.

Le système RAG améliore significativement la qualité des réponses grâce à l'intégration des exemples les plus similaires récupérés par le module de retrieval. Cette approche permet au modèle de s'appuyer sur un contexte pertinent avant de produire sa classification.

Les résultats obtenus démontrent l'efficacité de la combinaison Retrieval + LLM pour la détection du phishing. Toutefois, certaines catégories d'attaques, notamment les arnaques liées à la livraison (*Delivery Scam*), nécessitent davantage d'exemples dans la base de connaissances afin d'améliorer la généralisation du système.

3.15 Exemple de résultat généré

Pour le message suivant :

Your bank account has been suspended. Click here immediately : <http://bank-login-free.com>

Le système produit une sortie similaire :

Listing 3.8 : Exemple de sortie JSON

```
1 {
2   "classification": "phishing",
3   "risk_level": "high",
4   "detected_attack_type": "Banking Fraud",
5   "detected_elements": [
6     "Urgency",
7     "Sensitive Information Request",
8     "Suspicious URL"
9   ],
10  "url_analysis": [
11    {
12      "risk_score": 55,
13      "risk_level": "Medium",
14      "reasons": [
```

```

15         "Uses HTTP instead of HTTPS",
16         "Contains suspicious keyword: bank",
17         "Contains suspicious keyword: login",
18         "Contains suspicious keyword: free"
19     ]
20 }
21 ]
22 }

```

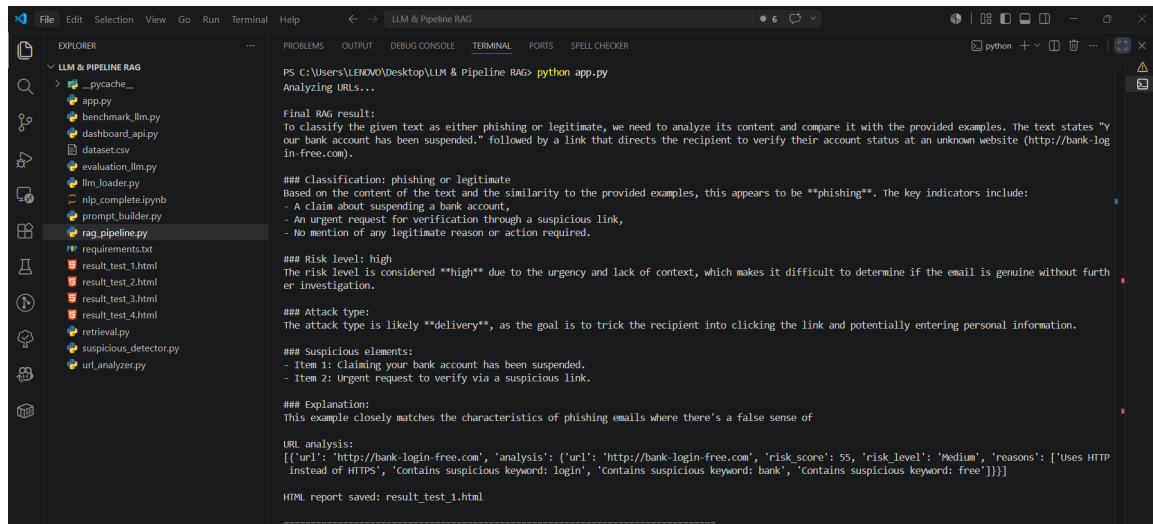


Fig. 3.7 : Exemple de résultat généré par le pipeline RAG

3.16 Intégration avec le dashboard

Afin de faciliter l'intégration avec le dashboard, une fonction API simple a été préparée. Elle permet d'envoyer un message au pipeline et de récupérer directement les résultats structurés.

Listing 3.9 : API pour le dashboard

```

1 from retrieval import retrieve_similar
2 from rag_pipeline import phishing_pipeline
3
4 def analyze_message(message):
5
6     docs = retrieve_similar(
7         message,
8         top_k=3
9     )
10
11     return phishing_pipeline(
12         message,
13         docs
14     )

```


Sortie exploitable par le dashboard

Cette fonction retourne les champs nécessaires pour l’affichage dans l’interface web : classification, niveau de risque, type d’attaque, éléments suspects, explication, recommandations, analyse des URLs et documents récupérés.

3.17 Preuves d’exécution

Afin de valider le fonctionnement du module, plusieurs captures d’écran ont été ajoutées : structure du projet, récupération des documents similaires, génération de la réponse par le LLM, détection des éléments suspects, analyse des URLs, benchmark des LLMs et résultats d’évaluation.

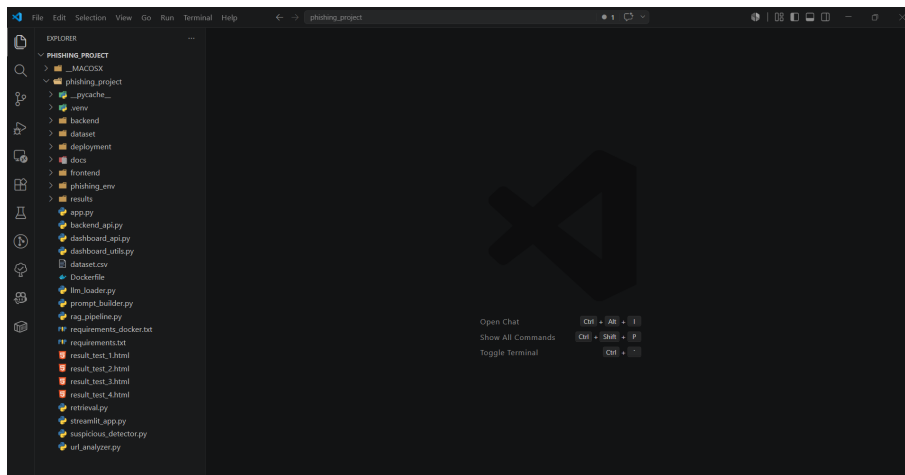


Fig. 3.8 : Structure des fichiers de la partie LLM et RAG

3.18 Limites et améliorations possibles

Limites du système

- certains LLMs ne respectent pas toujours le format imposé ;
- la précision peut être réduite par des faux positifs ;
- l’analyse des URLs reste basée sur des règles simples ;
- le système dépend fortement de la qualité du dataset ;
- le temps d’inférence peut être élevé sur CPU.

Améliorations possibles

- enrichir le dataset avec davantage d'exemples ;
- améliorer le prompt engineering ;
- ajouter une analyse WHOIS des URLs ;
- intégrer une analyse OCR pour les images de phishing ;
- optimiser le pipeline pour un déploiement plus rapide.

3.19 Conclusion

Le module LLM et RAG développé permet d'analyser les messages suspects de manière structurée et explicable. L'utilisation du retrieval permet de fournir au modèle de langage un contexte pertinent basé sur des exemples similaires. Le LLM peut alors générer une réponse plus fiable comprenant la classification, le niveau de risque, les éléments suspects et les recommandations.

Le benchmark réalisé a montré que [Qwen2.5](#) est le modèle le plus adapté à notre système, car il fournit des réponses cohérentes et respecte mieux le format attendu. Les modules complémentaires d'analyse d'URL et de détection des éléments suspects renforcent la capacité du système à expliquer ses décisions.

Références

1. Lewis P., Perez E., Piktus A., et al., *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*, NeurIPS, 2020.
2. Reimers N., Gurevych I., *Sentence-BERT : Sentence Embeddings using Siamese BERT-Networks*, EMNLP, 2019.
3. Johnson J., Douze M., Jégou H., *Billion-scale Similarity Search with FAISS*, IEEE Transactions on Big Data, 2019.
4. Robertson S., Zaragoza H., *The Probabilistic Relevance Framework : BM25 and Beyond*, Foundations and Trends in Information Retrieval, 2009.
5. Hugging Face Transformers Documentation.
6. Sentence Transformers Documentation.
7. FAISS Documentation.
8. PyTorch Documentation.
9. Streamlit Documentation.
10. Plotly Documentation.

Conclusion Générale

Ce projet a permis de concevoir et de mettre en œuvre une solution complète de détection et d'analyse du phishing reposant sur les technologies modernes du traitement automatique du langage naturel, de la recherche d'information et des modèles de langage génératifs.

Dans un premier temps, un dataset multilingue couvrant différents scénarios d'attaques de phishing a été construit et structuré afin de constituer une base de connaissances exploitable par le système. Cette étape a permis de disposer d'un corpus riche, équilibré et représentatif des menaces couramment rencontrées dans des contextes réels.

Par la suite, un moteur de retrieval hybride combinant FAISS et BM25 a été développé afin d'exploiter simultanément la similarité sémantique et la recherche par mots-clés. Les résultats expérimentaux ont montré que cette approche hybride améliore la pertinence des documents récupérés et fournit une base solide pour l'analyse des messages suspects.

Enfin, l'intégration d'une architecture Retrieval-Augmented Generation (RAG) associée à des modèles de langage a permis de produire des analyses détaillées, explicables et contextualisées. Le système développé est capable d'identifier les tentatives de phishing, d'estimer leur niveau de risque, de détecter les éléments suspects présents dans les messages et de générer des recommandations adaptées à l'utilisateur. Les évaluations réalisées ont démontré des performances encourageantes, avec une bonne capacité de généralisation sur différents types d'attaques et plusieurs langues.

Bien que les résultats obtenus soient satisfaisants, certaines pistes d'amélioration restent envisageables. Parmi celles-ci figurent l'enrichissement du dataset avec davantage de données réelles, l'utilisation de modèles d'embeddings plus spécialisés pour les langues multilingues, l'intégration de techniques avancées d'analyse d'URLs et d'images de phishing, ainsi que l'optimisation du pipeline pour un déploiement à grande échelle.

En conclusion, ce travail démontre l'intérêt de la combinaison entre les techniques de Retrieval et les modèles de langage pour renforcer la cybersécurité. L'approche proposée constitue une base solide pour le développement de systèmes intelligents capables d'assister efficacement les utilisateurs dans la détection précoce des menaces de phishing et la protection de leurs données sensibles.